

Pregunta 1

Finalizado

Se puntúa 2,00
sobre 2,00

Analice el siguiente bloque de código en Java:

```
public String calcular(int a) {  
    int val = 0;  
    while a < 10 {  
        val = val + "Z";  
        a++;  
    }  
    return true;  
}
```

¿Cuál es la naturaleza y cantidad aproximada de las fallas de este extracto?

- ☐ 1. **b) Es completamente válido de forma estructural y carece de errores sintácticos, sólo expone malas prácticas operativas lógicas en tiempo de ejecución.**
- ☒ 2. **a) Presenta 1 Error Sintáctico (ausencia de paréntesis obligatorios () en la condición del while), y 2 Errores Semánticos (intentar asignar un String en una variable int, y retornar un boolean cuando la firma exige String).**
- ☐ 3. **c) Consta de 3 Errores puramente Sintácticos puesto que la JVM validará el tipado dinámicamente recién en la memoria.**

Pregunta 2

Finalizado

Se puntúa 2,00
sobre 2,00

Si nos situamos estrictamente sobre el componente de "legibilidad" para un lenguaje de programación, ¿cuál definición refleja este atributo específico?

- ☒ 1. **c) La facilidad empírica con la cual un programador promedio es capaz de leer y comprender a simple intuición la funcionalidad de un código, agilizando todo trabajo posterior de detección de bugs y mantenimiento general.**
- ☐ 2. **a) El nivel óptimo con el que el código fuente escrito puede ser leído sintácticamente e interpretado a priori por el compilador, otorgándole la capacidad de generar de forma directa árbol abstractos de gran rendimiento.**
- ☐ 3. **b) Es la cualidad que fuerza al lenguaje a mantenerse cercano exclusivamente a la lógica matemática en estado crudo e inmutable, logrando minimizar un programa completo a un comando por línea sin perder fidelidad algorítmica.**

Pregunta 3

Finalizado

Se puntúa 2,00
sobre 2,00

Analice el siguiente pseudocódigo y asuma que el lenguaje ejecutado utiliza una cadena puramente dinámica:

```
Program Game;
```

```
var score: integer;
```

```
Procedure Play;
```

```
var score: integer;
```

```
begin
```

```
    score := 100;
```

```
    Show();
```

```
end;
```

```
Procedure Show;
```

```
begin
```

```
    write(score);
```

```
end;
```

```
begin
```

```
    score := 0;
```

```
    Play();
```

```
end;
```

¿Qué valor imprimirá la ejecución de la subrutina Show()?

- ☐ 1. a) Imprimiría 0, ya que en los procedimientos de Pascal siempre se enlaza la variable de la cabecera principal por defecto.
- ☒ 2. b) Imprimiría 100. La cadena dinámica busca la variable score basándose en el historial de invocación, y encuentra la variable activa más reciente dentro del entorno de Play (el procedimiento llamador).
- ☐ 3. c) Generaría un error en tiempo de ejecución, dado que Show es incapaz de localizar una variable sin recibirla explícitamente mediante parámetros.

Pregunta 4

Finalizado

Se puntúa 2,00
sobre 2,00

Usted es confrontado con este esquema dinámico clásico usando punteros Pascal-like:

01: Procedure TestDin();

02: var ptr: ^integer;

03: begin

04: new(ptr);

05: ptr[^] := 10;

06: dispose(ptr);

07: end;

Basado en las directrices de la cátedra, ¿cuál determina respectivamente el tiempo de vida del puntero y de la variable dinámica apuntada?

- ☒ 1. **b) El tiempo de vida de la variable puntero ptr abarca la ejecución general de la línea 03 a la línea 07, mientras que el de la variable apuntada corre exclusivamente entre la línea 04 y la línea 06.**
- ☐ 2. **a) El tiempo de vida del puntero ptr es de la línea 04 a la 06 inclusive, y el de la variable apuntada abarca todo el proceso de la línea 03 a la 07 atándose al tiempo estructural.**
- ☐ 3. **c) Ambas instancias comparten un tiempo de vida idéntico desde que inicia el bloque en la línea 03 hasta la terminación en la línea 07.**

Pregunta 5

Finalizado

Se puntúa 2,00
sobre 2,00

Observe la siguiente gramática BNF: $E ::= E + E \mid \text{numero}$. Indique su problema capital desde la validación léxico/sintáctica al generar el árbol y la forma de resolverlo:

- ☐ 1. a) No es capaz de tolerar símbolos matemáticos ya que el '+' es un terminal excluido por defecto de forma universal.
- ☐ 2. b) Carece de identificador inicio, lo cual siempre bloquea la construcción de grafos léxicos desde la derivación semántica estática.
- ☒ 3. c) Contiene simultáneamente recursividad tanto hacia la derecha como hacia la izquierda, lo que la vuelve totalmente ambigua (permite múltiples interpretaciones de derivación para un mismo texto); se corrige estratificando sus producciones introduciendo agrupaciones del tipo "Factor".